MCP Security Scanner

Final project results — complete evaluation summary

Noam Ben-Lulu · Bar David
Networked Software Systems Lab (NSSL), Technion · Supervisor: Sorana Fraier
Evaluation snapshot: **17 October 2026**

> **A working MCP security analysis system, with strong results on reviewed development cases and clearly identified limits on unfamiliar server implementations.**

The project grew from a two-threat prototype into a scanner with **16 threat categories and 25 stable rules**. It accepts a server file or project directory, inspects source without executing it, and produces findings with evidence, locations, and recommendations. Console, JSON, and standalone HTML reports support both automation and presentation. Eligible Python MCP03/MCP06 findings have conservative automatic fixes with dry-run, backups, and verification.

Full-project scorecard

Evidence track	What was measured	Precision	Recall	F1
Final scanner: development only	10 exposed batches; 290 artifacts; 377 reviewed labels	**99.5%**	**89.7%**	**94.3%**
Original reserved evaluation	Earlier frozen scanner; 33 artifacts; 50 labels	**92.9%**	**43.3%**	**59.1%**
New reserved evaluation	Final frozen scanner; 10 files; one MCP03 rule	**Undefined**	**0% (0/2)**	**0%**

Read these rows separately. The first row describes performance on cases already used for development; it is not an estimate of performance on unseen servers. The original reserve later became development round 008 and is already included in that first row. The new reserve revealed an unsupported handwritten JSON-RPC registration style. None of these rows should be added together as a single overall benchmark.

What the complete development result means

- **192 of 214 positive labels detected**, with **22 missed positives** remaining.
- **1 false alarm across 163 negative labels**; **162 negatives correctly rejected**.
- **8 categories and 11 rules evaluated**, out of the implemented 16 categories and 25 rules.
- **188 additional finding instances were outside reviewed label scope** and excluded from scoring. The 99.5% precision figure does not describe every emitted finding.
- **180 unit tests passed** in the publication checkout; recorded development scans had **zero scan errors across all 290 artifacts**. Software tests and scan completion are separate from detection effectiveness.

The strongest defensible conclusion is that the scanner performs well on its reviewed development set and provides a usable review workflow. Its generalization to unfamiliar registration styles remains limited. This report presents both the gains and the failures needed to assess that conclusion.

Understanding precision, recall, and the other metrics

Each scored decision asks whether a particular **rule** detects a labeled condition in an **artifact**, sometimes within a specified function. A label is not necessarily an independently confirmed exploitable vulnerability. Multiple findings in the same reviewed scope do not create extra true positives.

Confusion matrix: final scanner on all development batches

Actual reviewed label	Scanner flags the rule	Scanner does not flag the rule	Total
Positive: the labeled condition is present	**192 true positives (TP)**	**22 false negatives (FN)**	**214**
Negative: that condition is absent	**1 false positive (FP)**	**162 true negatives (TN)**	**163**
Total scored decisions	**193**	**184**	**377**

Metric	Question it answers	Formula and development result
Precision	When the scanner flags a reviewed condition, how often is it correct?	TP / (TP + FP) = 192 / 193 = **99.5%**
Recall / sensitivity	Of the real positive labels, how many does the scanner catch?	TP / (TP + FN) = 192 / 214 = **89.7%**
F1 score	How well are precision and recall balanced?	2TP / (2TP + FP + FN) = 384 / 407 = **94.3%**
Miss rate	What fraction of positive labels is missed?	FN / (TP + FN) = 22 / 214 = **10.3%**
False-positive rate	What fraction of negative labels triggers an incorrect alarm?	FP / (FP + TN) = 1 / 163 = **0.6%**
Specificity	How often are negative labels correctly left unflagged?	TN / (TN + FP) = 162 / 163 = **99.4%**
Accuracy	What fraction of all scored decisions is correct?	(TP + TN) / total = 354 / 377 = **93.9%**

For this project, we want higher recall and a lower miss rate. High precision means fewer false alarms within the scored scope; it does not imply that the scanner catches most real signals. A scanner can have high precision while missing many positives.

Why accuracy is not the headline: the new reserve has two positive and eight negative labels. Predicting no findings produces **80% accuracy while missing both positives**. That is why the presentation should show recall, precision, and their denominators together. With no positive predictions, precision is undefined, not 100%. F1 is zero here because positives were missed.

These are descriptive sample measurements, not population guarantees. The development labels are selected, the examples include related templates, and label review still needs independent human adjudication. The 188 unscored predictions enter none of these denominators.

All ten development rounds

The table uses **one final frozen scanner** on the original manifests for rounds 001–010. It reports the saved follow-up regression results, not a mixture of each round's original scanner version. All these cases are now exposed development data.

Round	Data / focus	Artifacts / labels	TP / FP / FN / TN	Precision	Recall	F1
001	Native MCP fixtures; stability and signatures	13 / 24	4 / 1 / 4 / 15	80.0%	50.0%	61.5%
002	Native fixtures; command and secret flows	18 / 27	14 / 0 / 0 / 13	100%	100%	100%
003	Native fixtures; metadata, execution, authentication	34 / 59	36 / 0 / 0 / 23	100%	100%	100%
004	Cisco fixtures; helpers, network, metadata behavior	46 / 71	44 / 0 / 0 / 27	100%	100%	100%
005	Adapted public metadata	78 / 78	45 / 0 / 3 / 30	100%	93.8%	96.8%
006	Synthetic paired source behaviors	24 / 24	12 / 0 / 0 / 12	100%	100%	100%
007	Adapted Vault metadata	20 / 20	9 / 0 / 1 / 10	100%	90.0%	94.7%
008	Consumed original reserve; native files	33 / 50	16 / 0 / 14 / 20	100%	53.3%	69.6%
009	Further related-family Vault records	12 / 12	6 / 0 / 0 / 6	100%	100%	100%
010	Synthetic source-flow pairs	12 / 12	6 / 0 / 0 / 6	100%	100%	100%
Development total	**Exposed cases; labeled scope only**	**290 / 377**	**192 / 1 / 22 / 162**	**99.5%**	**89.7%**	**94.3%**

The aggregation checks **290 distinct artifact hashes and 377 distinct, non-overlapping label scopes**. It counts each reviewed scope once, rather than adding repeated runs. Different source families, adapted examples, and synthetic pairs still have different difficulty and can be semantically related; the total is a descriptive development score. Round 004 uses its original 71 labels; three post-test controls in a separate repair report are not added to this total.

Measured engineering improvements

Paired development comparison	Improvement on the same reviewed cases
Guarded command execution, round 001 retest during round 002	False alarms **9** → **1**; precision **30.8%** → **80.0%**, with recall unchanged at 50%.
Helper/metadata repairs, original round 004 labels	Positive detections **10** → **44 of 44**; recall **22.7%** → **100%** after examining the failures.
Adapted metadata, round 005	Positive detections **2** → **45 of 48**; recall **4.2%** → **93.8%**; false alarms **2** → **0**.
Original reserve reused as round 008	Positive detections **13** → **16 of 30**; recall **43.3%** → **53.3%**; false alarms **1** → **0**.

These comparisons support specific repairs; they do not independently validate unseen performance. The last three rounds left every previously scored decision in rounds 001–007 unchanged, including their remaining misses and false alarm.

Where detection is stronger — and where it is weaker

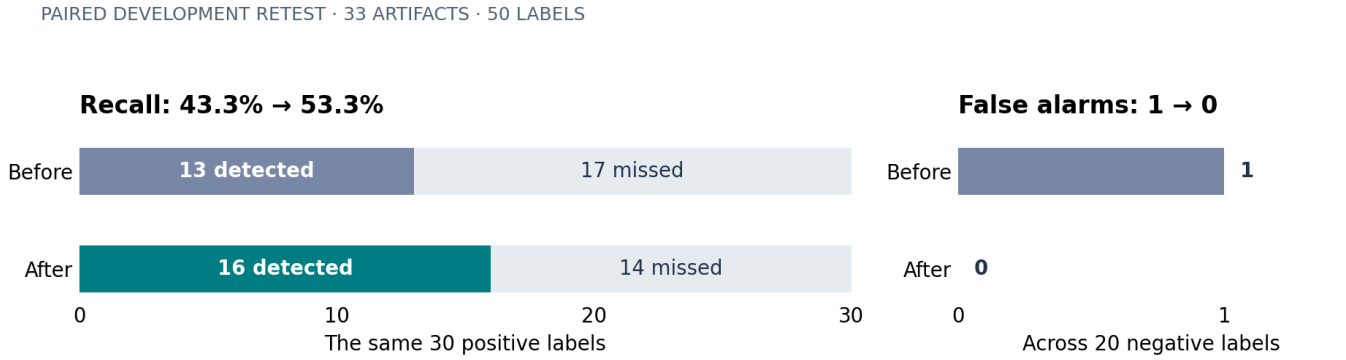
Final scanner, pooled development labels only. Counts make the sample size and remaining misses visible. Precision is undefined when the scanner made no scored positive predictions.

Category	Positive / negative labels	TP / FP / FN / TN	Precision	Recall
MCP01 · secrets	18 / 16	16 / 0 / 2 / 16	100%	88.9%
MCP02 · broad privileges	1 / 0	0 / 0 / 1 / 0	Undefined	0%
MCP03 · tool poisoning	95 / 67	89 / 1 / 6 / 66	98.9%	93.7%
MCP05 · command/code execution	64 / 54	58 / 0 / 6 / 54	100%	90.6%
MCP06 · untrusted context	10 / 5	6 / 0 / 4 / 5	100%	60.0%
MCP07 · authentication	10 / 4	9 / 0 / 1 / 4	100%	90.0%
MCP09 · confusable names	2 / 0	0 / 0 / 2 / 0	Undefined	0%
MCP13 · SSRF	14 / 17	14 / 0 / 0 / 17	100%	100%

The largest reviewed positive groups are tool poisoning and command/code execution. They show strong development recall, although poisoning includes many adapted text records. MCP06 retains four misses out of ten positives. The tiny MCP02 and MCP09 groups are entirely missed. MCP13's perfect development score covers only 14 positive labels and does not establish universal SSRF detection.

Coverage gap: MCP04, MCP08, MCP10, MCP11, MCP12, MCP14, MCP15, and MCP16 have no scored labels in this pooled ten-round set. Their implemented rules and unit tests are not equivalent to external effectiveness validation. Catalog breadth must not be presented as validated protection across all categories.

A concrete improvement on native server artifacts



The three recovered round 008 cases concern derived Python tool metadata, JavaScript dynamic code, and HTTP text returned by a Python tool. This is a **10 percentage-point recall increase** and **17.6% fewer misses** on those same 30 positive labels. The latest round 008 run also emitted **77 unscored finding instances**, excluded from its precision denominator.

Reserved and historical fresh tests

These results preserve what each frozen scanner achieved before tuning on the respective batch. They are essential evidence about generalization and must remain separate from the high development scores. Different rows use different scanner versions and data families; they are not a performance trend on a common test set.

Frozen evaluation	Artifacts / labels	TP / FP / FN / TN	Precision	Recall	F1
Historical fresh batch 002	18 / 27	4 / 0 / 10 / 13	100%	28.6%	44.4%
Historical fresh batch 003	34 / 59	24 / 3 / 12 / 20	88.9%	66.7%	76.2%
Historical fresh batch 004	46 / 71	10 / 0 / 34 / 27	100%	22.7%	37.0%
Original reserved run · 16 Oct	33 / 50	13 / 1 / 17 / 19	92.9%	43.3%	59.1%
New reserved run · 17 Oct	10 / 10	0 / 0 / 2 / 8	Undefined	0%	0%

All five runs had zero scan errors. Their unscored prediction counts were **3, 48, 16, 37, and 0**, respectively. Those findings are excluded from the precision denominators. In batch 004, subsequent investigation identified three false alarms among the unscored predictions; the original locked score was preserved. Zero scored false alarms therefore does not imply zero false alarms in all output.

Original reserve: promising precision, limited recall

The 33 artifacts came from **seven repositories**, with **30 positive and 20 negative labels** covering **eight categories and ten rules**. The scanner correctly flagged 13 of 14 scored positive predictions, but missed 17 of the 30 positive labels. This explains why recall became a priority. The later 53.3% recall on these same cases is development evidence, not a new reserved result.

New reserve: the extraction gap

Ten native MCP-DecayBench files at a pinned revision contained **two positive and eight negative MCP03 labels**. The scanner was frozen before source review; labels were recorded before predictions. The run saved hashes, predictions, a code snapshot, and a consumed marker. **Zero tools were recognized across all ten files.**

The tool-bearing fixtures expose metadata through handwritten JSON-RPC `tools/list` dispatch and module-level dictionaries. That registration style is outside the current extractor, so metadata never reached the poisoning detector. Both missed positives remain false negatives. Zero false alarms here does not establish reliable negative discrimination. One negative is a prompt-library control with no tools.

This small reserve covers one rule and a shared implementation family; it cannot estimate overall MCP recall. It nevertheless demonstrates a real coverage gap. Both reserves are now consumed. Repairs based on their failures require a newly reviewed reserve for the next unbiased evaluation. No scanner changes followed the latest reserved run.

Delivered system, practical evidence, and final assessment

Evidence beyond classification scores

Project outcome	Supporting evidence and its limit
Real-project processing	A recorded development check completed **10 public targets, 566 supported files, and 398 tools**. This demonstrates processing coverage, not 398 vulnerabilities or current detection accuracy.
Supported low-level registration	Recorded reference scans extracted **1 Fetch, 12 Git, and 2 Time tools** using supported SDK list/call handlers. Handwritten JSON-RPC remains a different, unsupported style.
Software verification	**172 tests at the scanner freeze; 180 after demo/report additions**, all passing in the publication checkout. Tests include parser, flow, redaction, reporting, and fixer behavior.
Remediation safety	Recorded checks confirm dry-run preservation, correct backups, valid Python, and idempotence. One sample went from **17 to 12 findings after two edits**: explicitly a partial fix, not complete protection.
Reproducibility	Pinned sources, label manifests, code/source hashes, saved predictions, code snapshots, and one-use reserve tracking. Deterministic evaluation; no LLM API calls or execution of target servers.

An earlier finding-level review recorded **28.9% precision**: 13 credible findings, 32 false positives, and 38 context-dependent findings excluded. It used a historical scanner and a different scoring unit, so it is not directly comparable to the current 99.5% development score.

Live demonstration

Run from the repository root in PowerShell:

```
python demo.py .\tests\samples\vulnerable_tool_poisoning.py --html scan-report.html
Start-Process .\scan-report.html
```

This performs a fresh scan and prints an HTML link. Replace the sample path with an existing server file or directory. Show one finding's source, rule, evidence, and recommendation. The intentionally vulnerable sample is a demonstration, not an accuracy benchmark. Exit codes: 0 no findings; 1 findings; 2 error/incomplete scan.

Final assessment and next priority

The scanner is a functioning review aid with strong measured performance on reviewed development cases. Reliable broad detection on unseen MCP implementations is not yet established. Remaining limitations include static and bounded reasoning, selected labels, adapted/synthetic records, related templates, independent human adjudication still pending, and incomplete runtime and whole-project validation. No controlled comparison establishes superiority over other scanners.

Next priorities: handwritten JSON-RPC extraction, remaining misses, independently reviewed labels, and a new reserve. **820 unadmitted text records** remain as candidate material, not independent tests. Present development and reserved results together, then demonstrate the workflow. A short slide-ready version accompanies this report.

Auditable data and sources

- Computed full-project metrics, round table CSV, and summary calculation script. The script audits labels and saved outcomes without rescanning a corpus.
- Saved final-candidate outcomes, development history, and individual before/after changes.
- Original validation report, follow-up report, and new reserved metrics.
- Historical operational check, earlier finding review, and fixer verification.